

VERICORE - ASIC VERIFICATION SERIES

30 Most Important UVM Interview Questions with Detailed Solutions

From Beginner to Advanced UVM Concepts

30

Questions

3

Difficulty Levels

10

Code Examples

BEGINNER

INTERMEDIATE

ADVANCED

Prepared by:

KITTU K PATEL

ASIC Verification | System Verilog | UVM Methodology

-
- For Educational Purposes Only
 - Version 1.0

Contents

1	Introduction to UVM	3
1.1	What is UVM?	3
1.2	Why UVM is Used in Industry	3
1.3	UVM Testbench Architecture	4
1.4	Career Opportunities in Verification	4
2	UVM Phase Flow	5
3	Beginner Level Questions	6
4	Intermediate Level Questions	13
5	Advanced Level Questions	21
6	Quick Reference: UVM Cheat Sheet	37
6.1	UVM Macro Reference.....	37
6.2	Interview Preparation Checklist.....	37
	Closing Note from VeriCore	38

1 Introduction to UVM

1.1 What is UVM?

Universal Verification Methodology (UVM) is a standardised, open-source verification methodology built on top of SystemVerilog. It provides a reusable, scalable, and structured framework for verifying complex digital hardware designs. UVM was standardised by Accellera in 2011 and later adopted as **IEEE 1800.2** in 2017.

Think of UVM as a *professional construction blueprint* — just as architects follow standard blueprints to build buildings consistently, verification engineers use UVM to build consistent, reusable testbenches.

Real Industry Use Case

Where is UVM Used?
Intel, Qualcomm, NVIDIA, AMD, Apple, MediaTek, Arm — every major semiconductor company uses UVM-based verification for their ASIC/SoC designs. It is the de facto standard in VLSI verification.

1.2 Why UVM is Used in Industry

Feature	Benefit
Reusability	Testbenches can be reused across projects with minimal changes
Scalability	Scales from simple block-level to complex chip-level verification
Standardisation	Common methodology across teams, companies, and tools
Coverage-Driven	Built-in support for functional coverage and constrained random
Factory Mechanism	Easy component substitution without modifying base code
TLM Communication	Clean, decoupled communication between components

1.3 UVM Testbench Architecture

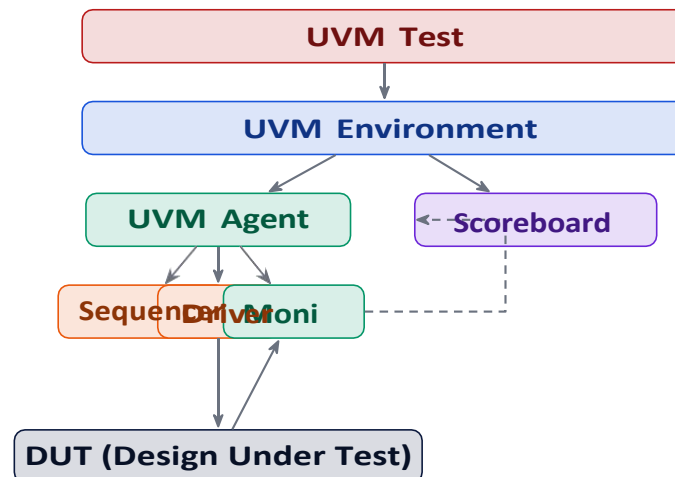


Figure 1. UVM Testbench Architecture Overview

1.4 Career Opportunities in Verification

The demand for skilled Verification Engineers is consistently high across the semiconductor industry. Key roles include:

DV Engineer — Design Verification using SystemVerilog & UVM

UVM Architect — Designing reusable testbench frameworks

Formal Verification Engineer — Property checking using tools like JasperGold

Emulation Engineer — Hardware-assisted verification using Palladium/Veloce

IP Verification Engineer — Block-level verification of reusable IPs

Average salaries range from **8–15 LPA** (freshers) to **25–60 LPA** (senior/architect level) in India, and **\$120K–\$200K+** in the US market.

2 UVM Phase Flow

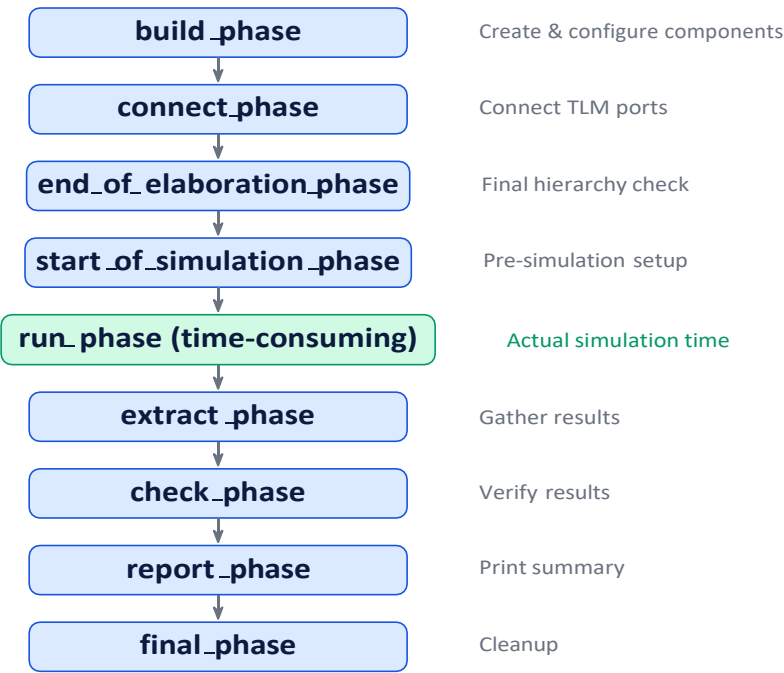


Figure 2. UVM Phase Execution Flow

3 Beginner Level Questions

What is UVM and why is it used in ASIC Verification?

BEGINNER

Short Answer: UVM (Universal Verification Methodology) is a standardised, class-library-based verification framework built on SystemVerilog. It provides reusable, scalable testbench components and is the industry standard for ASIC/SoC verification.

Detailed Explanation:

Before UVM, every verification team built their own testbench from scratch — leading to inconsistency, duplication of effort, and poor reusability. UVM solved this by providing:

A **standard base class library** (BCL) — all components extend from `uvm_component` or `uvm_object`

A **phasing mechanism** — structured execution from build to report phase

A **factory** — allows component substitution without changing test code

TLM ports — clean communication between components

A **configuration database** — pass configuration across hierarchy

Analogy: UVM is like a *restaurant franchise system*. Each restaurant (project) follows the same operating procedures (UVM structure), so staff can move between locations (teams) with minimal retraining.

UVM Minimal Testbench Skeleton

```

1 // UVM Test - top of the testbench hierarchy
2 class my_test extends uvm_test;
3   'uvm_component_utils(my_test) // Register with factory
4
5   my_env env; // Environment handle
6
7   // Constructor
8   function new(string name="my_test", uvm_component parent=null);
9     super.new(name, parent);
10  endfunction
11
12 // Build phase - create components
13 function void build_phase(uvm_phase phase);
14   super.build_phase(phase);
15   env = my_env::type_id::create("env", this);
16 endfunction
17
18 // Run phase - start stimulus
19 task run_phase(uvm_phase phase);
20   my_sequence seq;
21   phase.raise_objection(this); // Keep simulation alive
22   seq = my_sequence::type_id::create("seq");
23   seq.start(env.agent.sequencer);
24   phase.drop_objection(this); // Allow simulation to end
25 endtask
26 endclass

```

Pro Tip

Always use 'uvm_component utils for components and 'uvm_object utils for objects (transactions, sequences). This registers the class with the UVM factory — essential for override capability.

Key Takeaways

UVM = Standardised + Reusable + Scalable verification framework
Built on SystemVerilog; standardised as IEEE 1800.2
Every UVM component has a structured lifecycle via phases
Factory registration is mandatory for proper UVM usage

Explain UVM Component Hierarchy.**BEGINNER**

Short Answer: UVM components form a parent-child tree hierarchy rooted at uvm_top. Each component is created with a name and parent pointer, forming the testbench topology.

Component Hierarchy:

Class	Purpose
uvm_void	Root of all UVM classes
uvm_object	Base for all non-component objects (transactions, sequences)
uvm_component	Base for all structural components
uvm_test	Top-level test; controls the scenario
uvm_env	Container for agents, scoreboards, coverage
uvm_agent	Groups sequencer + driver + monitor
uvm_driver	Drives stimulus to DUT via interface
uvm_monitor	Observes DUT interface; sends to scoreboard
uvm_scoreboard	Compares expected vs actual results
uvm_sequencer	Controls sequence execution to driver

Interview Trick

Interviewers often ask: *"What is the difference between uvm_object and uvm_component?"*

Answer: uvm_component has a *fixed position* in the hierarchy (parent/child) and participates in phases. uvm_object has *no hierarchy* — it is created dynamically (like transactions). Sequences are uvm_object descendants, NOT components.

Common Mistakes

Confusing `uvm_sequence` with `uvm_component` — sequences are objects
Forgetting to pass parent in `::create()` — causes null parent issues
Using `new()` instead of `type_id::create()` — bypasses factory

What is the UVM Factory and why is it important?

BEGINNER

Short Answer: The UVM factory is a mechanism that allows creation and substitution of objects/components at runtime without modifying the original testbench code. It enables **overriding** classes for reuse and debug.

How it Works:

1. Register class with factory using `'uvm_component_utils` or `'uvm_object_utils`
2. Create instances using `type_id::create()` instead of `new()`
3. Override classes from the test level without changing environment code

UVM Factory Override Example

```
1 // Base driver
2 class base_driver extends uvm_driver #(my_txn);
3     'uvm_component_utils (base_driver)
4     // ... normal driver code
5 endclass
6
7 // Extended driver for error injection
8 class error_driver extends base_driver;
9     'uvm_component_utils (error_driver)
10    // Override run_phase to inject errors
11    task run_phase(uvm_phase phase);
12        // Force errors on the bus
13        super.run_phase ( phase );
14    endtask
15 endclass
16
17 // In the test - override without changing env/agent
18 class error_test extends base_test;
19     function void build_phase(uvm_phase phase);
20        // Override base_driver with error_driver globally
21        base_driver :: type_id :: set_type_override (
22            error_driver :: get_type ());
23        super.build_phase ( phase );
24    endfunction
25 endclass
```

Pro Tip

The factory is UVM's most powerful feature for **reusability**. Always create components and objects via factory (`type_id::create`) — never with `new()`. Instance

overrides (set inst override) override a specific path; type overrides affect all instances of that type.

What is uvm_config_db and how is it used?

BEGINNER

Short Answer: uvm_config_db is UVM's global configuration database. It is used to pass configuration values (virtual interfaces, integers, objects) from one component to another without tight coupling.

Syntax:

Set: uvm_config_db#(type)::set(context, path, key, value)

Get: uvm_config_db#(type)::get(context, "", key, variable)

uvm_config_db: Passing Virtual Interface

```
1 // In top-level module / test (SET side)
2 module tb_top;
3     my_if vif(); // Instantiate interface
4     dut u_dut(.clk(vif.clk), .data(vif.data));
5
6     initial begin
7         // Pass virtual interface into config_db
8         uvm_config_db #(virtual my_if)::set(
9             null, // Context (null = global)
10            "uvm_test_top.env.agent.driver", // Path
11            "vif", // Key name
12            vif // Value
13        );
14        run_test("my_test");
15    end
16 endmodule
17
18 // In Driver (GET side)
19 class my_driver extends uvm_driver #(my_txn);
20     virtual my_if vif; // Virtual interface handle
21
22     function void build_phase(uvm_phase phase);
23         super.build_phase(phase);
24         // Retrieve interface from config_db
25         if(!uvm_config_db #(virtual my_if)::get(
26             this, "", "vif", vif))
27             'uvm_fatal("NO_VIF", "Virtual interface not found!")
28     endfunction
29 endclass
```

Common Mistakes

Using wrong path string in set() — always verify full hierarchy path
Not checking return value of get() — always use if(!get()) with 'uvm_fatal
Setting config after build phase — set before or during build phase

Explain UVM Sequence, Sequencer, and Driver relationship.

BEGINNER

Short Answer: Sequence generates stimulus (transactions), Sequencer arbitrates between sequences, and Driver consumes transactions and drives them to the DUT. They communicate via the **TLM pull mechanism**.

Flow:



Sequence-Sequencer-Driver Handshake

```

1 // SEQUENCE - generates transactions
2 class my_sequence extends uvm_sequence #(my_txn);
3   'uvm_object_utils ( my_sequence )
4
5   task body();
6     my_txn txn;
7     repeat(10) begin
8       txn = my_txn::type_id::create("txn");
9       start_item (txn);           // Request sequencer access
10      if(!txn.randomize())        // Randomize transaction
11        'uvm_fatal("RAND_FAIL","Randomization failed")
12      finish_item (txn);          // Send to driver & wait
13    end
14  endtask
15 endclass
16
17 // DRIVER - consumes transactions
18 class my_driver extends uvm_driver #(my_txn);
19   'uvm_component_utils ( my_driver )
20   virtual my_if vif;
21
22   task run_phase(uvm_phase phase);
23     forever begin
24       my_txn txn;
25       seq_item_port.get_next_item (txn); // Pull from sequencer
26       drive_txn (txn);                  // Drive to DUT
27       seq_item_port.item_done ();       // Signal completion
28     end
29   endtask
30
31   task drive_txn(my_txn txn);
32     @(posedge vif.clk);
33     vif.data  <= txn.data;
34     vif.valid <= 1'b1;
35     @(posedge vif.clk);
36     vif.valid <= 1'b0;
37   endtask
38 endclass
  
```

Interview Trick

Interviewers love this question: *"What happens if you forget item done()?"*
 The sequencer will **deadlock** — it waits for the driver to signal completion before sending the next item. Always call item done() after processing each transaction.

What is a UVM Monitor and what is its role?**BEGINNER**

Short Answer: The UVM Monitor is a **passive** component that observes the DUT interface, collects transactions, and broadcasts them to scoreboards and coverage collectors via analysis ports. It does NOT drive signals.

Key Characteristics:

- Purely observational — never drives signals
- Reconstructs transactions from pin-level activity
- Sends collected transactions via uvm analysis_port
- Can have both input (DUT-side) and output (reference-side) monitors

UVM Monitor Implementation

```

1 class my_monitor extends uvm_monitor;
2   'uvm_component_utils(my_monitor)
3
4   // Analysis port to broadcast collected transactions
5   uvm_analysis_port #(my_txn) ap;
6   virtual my_if vif;
7
8   function void build_phase(uvm_phase phase);
9     super.build_phase(phase);
10    ap = new("ap", this); // Create analysis port
11    uvm_config_db #(virtual my_if)::get(this, "", "vif", vif);
12  endfunction
13
14  task run_phase(uvm_phase phase);
15    my_txn txn;
16    forever begin
17      // Wait for valid transaction on interface
18      @(posedge vif.clk iff vif.valid === 1'b1);
19      txn = my_txn::type_id::create("txn");
20      // Capture interface signals into transaction
21      txn.data = vif.data;
22      txn.address = vif.address;
23      // Broadcast to all connected subscribers
24      ap.write(txn);
25    end
26  endtask
27 endclass

```

What are UVM Phases and why are they important?

BEGINNER

Short Answer: UVM phases are a structured sequence of functions and tasks that control the lifecycle of a simulation — from component creation (build_phase) to final cleanup (final phase). They ensure all components initialise and connect in the correct order.

Phase	Type	Purpose
build_phase	function	Create child components
connect_phase	function	Connect TLM ports
end_of_elaboration	function	Final hierarchy checks
start_of_simulation	function	Print topology, config
run_phase	task	Actual simulation (time)
extract_phase	function	Collect results
check_phase	function	Compare/verify
report_phase	function	Print summary
final_phase	function	Cleanup

Pro Tip

Build phase executes top-down (parent before child). All other phases execute **bottom-up** (children before parent). This ordering ensures parent creates children before children try to connect. A very common interview question!

What is a UVM Scoreboard and how does it work?

BEGINNER

Short Answer: The UVM Scoreboard is a verification component that compares the DUT's actual output against expected (reference model) output. It implements `uvm_analysis_imp` to receive transactions from monitors.

UVM Scoreboard with Two Analysis Imports

```

1 'uvm_analysis_imp_decl(_expected) // Macro for named imp
2 'uvm_analysis_imp_decl(_actual)
3
4 class my_scoreboard extends uvm_scoreboard;
5     'uvm_component_utils(my_scoreboard)
6
7     // Two analysis imports - one for ref model, one for DUT
8     uvm_analysis_imp_expected #(my_txn, my_scoreboard) exp_imp;
9     uvm_analysis_imp_actual #(my_txn, my_scoreboard) act_imp;
10
11     my_txn exp_q[$]; // Queue for expected transactions
12     int pass_cnt, fail_cnt;

```

```
13
14 function void build_phase(uvm_phase phase);
15     super.build_phase(phase);
16     exp_imp = new("exp_imp", this);
17     act_imp = new("act_imp", this);
18 endfunction
19
20 // Called when reference model writes a transaction
21 function void write_expected(my_txn txn);
22     exp_q.push_back(txn);
23 endfunction
24
25 // Called when DUT monitor writes a transaction
26 function void write_actual(my_txn txn);
27     my_txn exp_txn;
28     if(exp_q.size() == 0) begin
29         'uvm_error("SB","Actual received but queue empty!")
30         return;
31     end
32     exp_txn = exp_q.pop_front();
33     // Compare fields
34     if(txn.data === exp_txn.data) begin
35         'uvm_info("SB", $sformatf("PASS: data =%0h", txn.data), UVM_LOW)
36         pass_cnt++;
37     end else begin
38         'uvm_error("SB", $sformatf(
39             "FAIL: exp=%0h got=%0h", exp_txn.data, txn.data))
40         fail_cnt++;
41     end
42 endfunction
43
44 function void report_phase(uvm_phase phase);
45     'uvm_info("SB", $sformatf(
46         "RESULT: PASS=%0d FAIL=%0d", pass_cnt, fail_cnt), UVM_NONE)
47 endfunction
48 endclass
```

4 Intermediate Level Questions

Explain TLM (Transaction Level Modeling) in UVM.

INTERMEDIATE

Short Answer: TLM provides abstract communication channels between UVM components. Instead of signal-level communication, components exchange complete transactions (objects), making testbenches more modular and reusable.

TLM Port Types:

Port Type	Direction	Use Case
uvm analysis_port	Out (1-to-many)	Monitor broadcasting
uvm analysis_imp	In (subscriber)	Scoreboard receiving
uvm blocking_put_port	Out	Sequence to sequencer
uvm blocking_get_port	In	Driver pulling from sequencer
uvm tlm_fifo	Bidirectional	Decoupled buffering

TLM Analysis Port Connection

```

1 // Monitor - has analysis port (publisher)
2 class my_monitor extends uvm_monitor;
3     uvm_analysis_port #(my_txn) ap; // 1 port, N subscribers
4
5     function void build_phase(uvm_phase phase);
6         ap = new("ap", this);
7     endfunction
8 endclass
9
10 // Scoreboard - has analysis_imp (subscriber)
11 class my_scoreboard extends uvm_scoreboard;
12     uvm_analysis_imp #(my_txn, my_scoreboard) analysis_imp;
13
14     function void write(my_txn txn);
15         // Automatically called when ap.write() is invoked
16         // Process incoming transaction
17     endfunction
18 endclass
19
20 // Environment - connect them
21 class my_env extends uvm_env;
22     my_monitor mon;
23     my_scoreboard sb;
24
25     function void connect_phase(uvm_phase phase);
26         // Connect monitor's port to scoreboard's imp
27         mon.ap.connect(sb.analysis_imp);
28     endfunction
29 endclass

```

Interview Trick

"What is the difference between uvm_analysis_port and uvm_blocking_put_port?"
uvm_analysis_port is a **broadcast** port (1 writer, N readers), non-blocking.
uvm_blocking_put_port is **point-to-point** and blocks until the receiver is ready.
Analysis ports are used for monitoring; put/get ports for data flow control.

What is a Virtual Interface in UVM and why is it needed?

INTERMEDIATE

Short Answer: A virtual interface is a handle to a SystemVerilog interface. It allows class-based UVM components (which live in dynamic space) to access the static interface signals of the DUT.

Why Needed: SystemVerilog interfaces are static (module-like). UVM classes are dynamic objects. A virtual interface bridges these two worlds.

Virtual Interface Declaration and Usage

```
1 // Step 1: Define interface (in module space - static)
2 interface my_if(input logic clk);
3     logic        valid;
4     logic [31:0] data;
5     logic [7:0]   address;
6     logic        ready;
7
8     // Clocking block for driver (synchronous)
9     clocking driver_cb @(posedge clk);
10        output valid, data, address;
11        input  ready;
12    endclocking
13
14    // Clocking block for monitor (observe)
15    clocking monitor_cb @(posedge clk);
16        input valid, data, address, ready;
17    endclocking
18 endinterface
19
20 // Step 2: In driver (class space - dynamic)
21 class my_driver extends uvm_driver #(my_txn);
22     virtual my_if vif;    // "virtual" = handle to interface
23
24     task drive_txn(my_txn txn);
25         // Access interface through virtual handle
26         @(vif.driver_cb);
27         vif.driver_cb.valid    <= 1'b1;
28         vif.driver_cb.data     <= txn.data;
29         vif.driver_cb.address <= txn.address;
30     endtask
31 endclass
```

What is UVM Objection Mechanism?

INTERMEDIATE

Short Answer: The UVM objection mechanism controls when the simulation ends during run_phase. A component raises an objection to prevent simulation from ending, and drops it when its work is done. When all objections are dropped,

simulation proceeds to the next phase.

Think of it as: A meeting room booking system. As long as someone has a booking (objection), the room (simulation) stays active. When everyone checks out (drops objection), the room is freed.

UVM Objection Usage

```

1 class my_test extends uvm_test;
2   task run_phase(uvm_phase phase);
3     my_sequence seq;
4
5     // RAISE objection - keep simulation alive
6     phase.raise_objection(this, "Starting test sequence");
7
8     seq = my_sequence::type_id::create("seq");
9     seq.start(env.agent.sequencer);
10
11    // Optional: drain time for DUT to respond
12    #100 ns;
13
14    // DROP objection - allow simulation to proceed
15    phase.drop_objection(this, "Test sequence complete");
16  endtask
17 endclass
18
19 // Set objection drain time (waits N ns after last drop)
20 function void end_of_elaboration_phase(uvm_phase phase);
21   uvm_objection obj = phase.get_objection();
22   obj.set_drain_time(this, 200 ns);
23 endfunction

```

Common Mistakes

- Raising objection inside a forked thread without proper join — can cause premature end
- Not dropping objection — simulation runs forever
- Dropping before DUT has responded — missing output checking

What is Functional Coverage in UVM? How is it implemented?

INTERMEDIATE

Short Answer: Functional coverage measures whether specific design scenarios (features) have been exercised during simulation. It is defined using SystemVerilog covergroup inside UVM components or subscriber classes.

Functional Coverage in UVM Subscriber

```

1 // Coverage collector - separate component
2 class my_coverage extends uvm_subscriber #(my_txn);
3   'uvm_component_utils ( my_coverage )

```



```

4
5  my_txn txn; // Current transaction
6
7  // Covergroup definition
8  covergroup txn_cg;
9      // Cover all address ranges
10     cp_addr: coverpoint txn.address {
11         bins low_addr = {[8'h00:8'h3F]};
12         bins mid_addr = {[8'h40:8'hBF]};
13         bins high_addr = {[8'hC0:8'hFF]};
14     }
15     // Cover read and write operations
16     cp_op: coverpoint txn.op {
17         bins read = {READ};
18         bins write = {WRITE};
19     }
20     // Cross coverage: every op at every address range
21     cx_addr_op: cross cp_addr, cp_op;
22 endgroup
23
24 function new(string name, uvm_component parent);
25     super new(name, parent);
26     txn_cg = new(); // Instantiate covergroup
27 endfunction
28
29 // Called automatically when monitor writes
30 function void write(my_txn t);
31     txn = t;
32     txn_cg.sample(); // Sample coverage
33 endfunction
34
35 function void report_phase(uvm_phase phase);
36     'uvm_info("COV", $sformatf(
37         "Functional Coverage: %.2f%%", txn_cg.get_coverage()),
38         UVM_NONE)
39 endfunction
40 endclass

```

Pro Tip

Separate coverage from the scoreboard into its own uvm subscriber class. This keeps concerns separated and allows enabling/disabling coverage without affecting checking logic. Connect both to the same monitor analysis port.

Explain Sequence Arbitration in UVM.**INTERMEDIATE**

Short Answer: When multiple sequences run concurrently on the same sequencer, arbitration determines which sequence gets access to the driver first. UVM supports several arbitration algorithms.

Algorithm	Description
SEQ_ARB_FIFO	Default; first-come first-served
SEQ_ARB_WEIGHTED	Weight-based priority
SEQ_ARB_RANDOM	Random selection
SEQ_ARB_STRICT_FIFO	Strict priority, FIFO within same priority
SEQ_ARB_STRICT_RANDOM	Strict priority, random within same priority
SEQ_ARB_USER	Custom user-defined arbitration

Setting Sequencer Arbitration

```

1 class priority_test extends base_test;
2   task run_phase(uvm_phase phase);
3     high_priority_seq hi_seq;
4     low_priority_seq lo_seq;
5
6     phase.raise_objection(this);
7
8     // Set weighted arbitration
9     env.agent.sequencer.set_arbitration(
10       UVM_SEQ_ARB_WEIGHTED);
11
12     // Fork both sequences simultaneously
13     fork
14       begin
15         hi_seq = high_priority_seq::type_id::create("hi");
16         hi_seq.set_priority(200); // Higher weight
17         hi_seq.start(env.agent.sequencer);
18       end
19       begin
20         lo_seq = low_priority_seq::type_id::create("lo");
21         lo_seq.set_priority(50); // Lower weight
22         lo_seq.start(env.agent.sequencer);
23       end
24     join
25
26     phase.drop_objection(this);
27   endtask
28 endclass

```

What are UVM Callbacks and when should they be used?

INTERMEDIATE

Short Answer: UVM callbacks allow users to inject custom behaviour into existing components without modifying their source code. They are hooks that can be registered and triggered at specific points in a component's execution.

Use Cases:

Error injection into a driver without modifying the driver class

Adding protocol-specific checks to a base monitor
Collecting additional coverage from existing components

UVM Callback Implementation

```

1 // Step 1: Define callback class
2 class driver_callbacks extends uvm_callback;
3   'uvm_object_utils (driver_callbacks)
4
5   // Callback hook - called before driving
6   virtual task pre_drive(my_driver drv, my_txn txn);
7     // Default: do nothing (override to customise)
8   endtask
9
10  virtual task post_drive(my_driver drv, my_txn txn);
11  endtask
12 endclass
13
14 // Step 2: Register callback in driver
15 class my_driver extends uvm_driver #(my_txn);
16   'uvm_register_cb (my_driver, driver_callbacks)
17
18   task run_phase(uvm_phase phase);
19     my_txn txn;
20     forever begin
21       seq_item_port.get_next_item (txn);
22       // Execute pre-drive callbacks
23       'uvm_do_callbacks (my_driver, driver_callbacks,
24                         pre_drive (this, txn))
25       drive_txn (txn);
26       // Execute post-drive callbacks
27       'uvm_do_callbacks (my_driver, driver_callbacks,
28                         post_drive (this, txn))
29       seq_item_port.item_done ();
30     end
31   endtask
32 endclass
33
34 // Step 3: Create error-injection callback
35 class error_inject_cb extends driver_callbacks;
36   'uvm_object_utils (error_inject_cb)
37
38   task pre_drive(my_driver drv, my_txn txn);
39     // Corrupt data with 10% probability
40     if ($urandom_range (0,9) == 0)
41       txn.data = ~txn.data; // Bit-flip!
42   endtask
43 endclass
44
45 // Step 4: Register in test
46 class error_test extends base_test;
47   function void build_phase(uvm_phase phase);
48     super.build_phase (phase);
49     error_inject_cb cb = error_inject_cb::type_id::create ("cb");
50     uvm_callbacks #(my_driver, driver_callbacks)::add (
51       env.agent.driver, cb);
52   endfunction

```

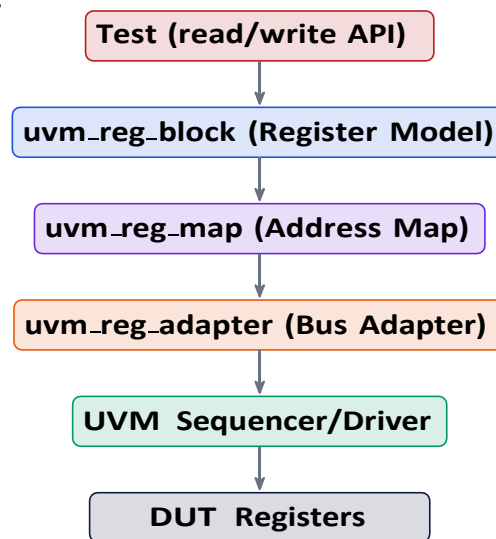
```
53 endclass
```

What is UVM Register Abstraction Layer (RAL)?

INTERMEDIATE

Short Answer: UVM RAL (Register Abstraction Layer) provides a high-level, object-oriented model for accessing DUT registers. It abstracts the physical bus interface, allowing register reads and writes via simple API calls instead of raw bus transactions.

RAL Architecture:



UVM RAL Register Model

```

1 // Define a register
2 class ctrl_reg extends uvm_reg;
3   'uvm_object_utils(ctrl_reg)
4
5 // Define fields
6   uvm_reg_field enable;
7   uvm_reg_field mode;
8   uvm_reg_field reserved;
9
10  function new(string name="ctrl_reg");
11    super.new(name, 32, UVM_NO_COVERAGE);
12  endfunction
13
14  function void build();
15    enable = uvm_reg_field::type_id::create("enable");
16    mode = uvm_reg_field::type_id::create("mode");
17    reserved = uvm_reg_field::type_id::create("reserved");
18
19    // configure(parent, size, lsb_pos, access, volatile,
20    //           reset_val, has_reset, is_rand, ind_accessible)
21    enable.configure(this, 1, 0, "RW", 0, 1'b0, 1, 1, 0);
  
```

```

22     mode.configure (    this, 3,  1, "RW", 0, 3'b0,  1, 1, 0);
23     reserved.configure (this, 28, 4, "RO", 0, 28'h0, 1, 0, 0);
24     endfunction
25 endclass
26
27 // In test - using RAL API
28 class ral_test extends base_test;
29     task run_phase(uvm_phase phase);
30         uvm_status_e  status;
31         uvm_reg_data_t data;
32         phase.raise_objection (this);
33
34         // Write to register using RAL
35         regmodel.ctrl.write(status, 32'h0000_0003);
36
37         // Read back and verify
38         regmodel.ctrl.read(status, data);
39         `uvm_info("RAL",$sformatf("CTRL = %0h", data), UVM_LOW)
40
41         // Field-level access
42         regmodel.ctrl.enable.set(1'b1);
43         regmodel.ctrl.update(status);    // Write only changed fields
44
45         phase.drop_objection (this);
46     endtask
47 endclass

```

5 Advanced Level Questions

Explain Factory Override with Instance and Type Override.

ADVANCED

Short Answer: UVM factory supports two override mechanisms — **type override** (replaces all instances of a class) and **instance override** (replaces only specific hierarchy paths).

Override Type	Scope	Use Case
Type Override	All instances globally	Replace driver for all agents
Instance Override	Specific hierarchy path	Replace driver only in agent A

Type vs Instance Override

```

1 class multi_agent_test extends base_test;
2     function void build_phase(uvm_phase phase);
3
4         // TYPE OVERRIDE - affects ALL base_driver instances
5         base_driver :: type_id :: set_type_override (

```

```

6         fast_driver :: get_type ();
7
8         // INSTANCE OVERRIDE - only for agent_b's driver
9         // Path: "uvm_test_top.env.agent_b.driver"
10        factory.set_inst_override_by_type (
11            base_driver :: get_type (),
12            slow_driver :: get_type (),
13            "uvm_test_top.env.agent_b.*");
14
15        super.build_phase ( phase ); // Creates components AFTER
            overrides
16
17    endfunction
18 endclass
19
20 // Print factory overrides for debug
21 function void end_of_elaboration_phase (uvm_phase phase);
22 // Prints all registered overrides
23 factory.print ();
24 // Debug a specific component creation
25 factory.debug_create_by_type (
26     base_driver :: get_type (), "uvm_test_top.env.agent_a");
27 endfunction

```

Interview Trick

Critical rule: Always set factory overrides **before** calling super.build phase() — because super.build phase() triggers child component creation. If you override after creation, it has no effect!

What is a Virtual Sequence and when is it used?

ADVANCED

Short Answer: A virtual sequence is a sequence that orchestrates multiple sub-sequences running on different sequencers simultaneously. It does not have its own sequencer but holds handles to multiple sequencers, enabling coordinated multi-agent stimulus generation.

When to Use: Multi-interface designs where stimulus must be synchronised across interfaces (e.g., AXI master + AXI slave coordination, PCIe TX + RX coordination).

Virtual Sequence Implementation

```

1 // Virtual sequence - coordinates multiple agents
2 class system_virtual_seq extends uvm_sequence;
3     'uvm_object_utils ( system_virtual_seq )
4     'uvm_declare_p_sequencer ( virtual_sequencer )
5
6     // Handles to actual sequencers
7     axi_master_sequencer axi_seqr;
8     apb_sequencer         apb_seqr;
9     ahb_sequencer         ahb_seqr;

```

```

10
11 task body();
12     axi_write_seq axi_seq ;
13     apb_config_seq apb_seq ;
14     ahb_read_seq ahb_seq;
15
16     // Get sequencer handles from virtual sequencer
17     axi_seqr = p_sequencer.axi_seqr;
18     apb_seqr = p_sequencer.apb_seqr;
19     ahb_seqr = p_sequencer.ahb_seqr;
20
21     // Step 1: Configure DUT via APB (serial)
22     apb_seq = apb_config_seq::type_id::create("apb_seq");
23     apb_seq.start(apb_seqr);
24
25     // Step 2: Run AXI write + AHB read in parallel
26     fork
27         begin
28             axi_seq = axi_write_seq::type_id::create("axi");
29             axi_seq.start(axi_seqr);
30         end
31         begin
32             ahb_seq = ahb_read_seq::type_id::create("ahb");
33             ahb_seq.start(ahb_seqr);
34         end
35     join
36 endtask
37 endclass
38
39 // Virtual Sequencer - just holds handles
40 class virtual_sequencer extends uvm_sequencer;
41     'uvm_component_utils(virtual_sequencer)
42     axi_master_sequencer axi_seqr;
43     apb_sequencer apb_seqr;
44     ahb_sequencer ahb_seqr;
45 endclass

```

How do you debug UVM simulations effectively?

ADVANCED

Short Answer: UVM provides built-in debugging mechanisms including verbosity control, 'uvm.info/'uvm.error reporting, factory print, topology print, and phase debug. Industry tools like Questa, VCS, and Xcelium add waveform and transaction-level debug.

Key Debug Techniques:

UVM Debug Commands and Techniques

```

1 // 1. Control verbosity at runtime
2 +UVM_VERBOSITY=UVM_HIGH // Command line
3 +UVM_VERBOSITY=UVM_DEBUG

```

```

4
5 // 2. Print testbench topology
6 function void end_of_elaboration_phase(uvm_phase phase);
7     uvm_top.print_topology(); // Prints full component tree
8     factory.print();         // Prints factory overrides
9 endfunction
10
11 // 3. Set verbosity per component
12 function void start_of_simulation_phase(uvm_phase phase);
13     env.agent.driver.set_report_verbosity_level(UVM_HIGH);
14 endfunction
15
16 // 4. UVM messaging macros
17 `uvm_info("TAG", "Message", UVM_LOW) // Informational
18 `uvm_warning("TAG", "Warning message") // Potential issue
19 `uvm_error("TAG", "Error - continue") // Error, keep running
20 `uvm_fatal("TAG", "Fatal - stop now") // Fatal, stop sim
21
22 // 5. Transaction-level debug
23 class my_txn extends uvm_sequence_item;
24     function void do_print(uvm_printer printer);
25         super.do_print(printer);
26         printer.print_field("data", data, 32, UVM_HEX);
27         printer.print_field("address", address, 8, UVM_HEX);
28         printer.print_string("op", op.name());
29     endfunction
30 endclass
31
32 // 6. Phase debug - set breakpoints
33 // +UVM_PHASE_TRACE // Trace all phase transitions
34
35 // 7. Config DB debug
36 // +UVM_CONFIG_DB_TRACE // Trace all set/get operations

```

Pro Tip

Always implement `do_print()`, `do_copy()`, and `do_compare()` in your transaction classes. These are called by UVM's built-in print, copy, and compare utilities and make debugging dramatically easier.

Explain UVM RAL Adapter and Predictor.**ADVANCED**

Short Answer: The RAL adapter translates UVM register operations (read/write) into bus-specific sequence items. The predictor intercepts bus transactions and updates the register model's mirror values automatically.

RAL Adapter and Predictor

```

1 // APB Bus Adapter for RAL
2 class apb_reg_adapter extends uvm_reg_adapter;
3   'uvm_object_utils ( apb_reg_adapter )
4
5   // Convert register operation -> bus transaction
6   function uvm_sequence_item reg2bus(
7       const ref uvm_reg_bus_op rw);
8       apb_txn txn = apb_txn::type_id::create("txn");
9       txn.paddr = rw.addr;
10      txn.pwdata = rw.data;
11      txn.pwrite = (rw.kind == UVM_WRITE) ? 1 : 0;
12      return txn;
13  endfunction
14
15  // Convert bus response -> register result
16  function void bus2reg(
17      uvm_sequence_item bus_item,
18      ref uvm_reg_bus_op rw);
19      apb_txn txn;
20      $cast(txn, bus_item);
21      rw.data = txn.prdata;
22      rw.status = UVM_IS_OK;
23  endfunction
24 endclass
25
26 // Connect in environment
27 class my_env extends uvm_env;
28   apb_reg_adapter adapter;
29   uvm_reg_predictor #(apb_txn) predictor;
30   my_reg_block regmodel;
31
32   function void connect_phase(uvm_phase phase);
33       // Connect register model to bus sequencer
34       regmodel.default_map.set_sequencer(
35           agent.sequencer, adapter);
36
37       // Connect predictor to monitor
38       agent.monitor.ap.connect(predictor.bus_in);
39       predictor.map = regmodel.default_map;
40       predictor.adapter = adapter;
41   endfunction
42 endclass

```

What is Phase Jumping in UVM?**ADVANCED**

Short Answer: Phase jumping allows a component to jump to a different phase during execution — either forward (skipping phases) or backward (re-running phases). It is used for specific recovery or re-initialisation scenarios.

Note: Phase jumping is rarely used in practice and should be approached with

caution as it can complicate testbench behaviour. Most interviewers ask this to test deep UVM knowledge.

Phase Jumping Example

```

1 class phase_jump_test extends base_test;
2   task run_phase(uvm_phase phase);
3     uvm_phase reset_ph;
4     phase.raise_objection(this);
5
6     // Run normal test sequence
7     run_normal_seq();
8
9     // Detect error condition - jump back to reset
10    if(error_detected) begin
11      reset_ph = uvm_reset_phase::get();
12      // Jump back to reset phase
13      phase.jump(reset_ph);
14      'uvm_info("JUMP","Jumping back to reset phase",UVM_LOW)
15    end
16
17    phase.drop_objection(this);
18  endtask
19 endclass

```

How do you implement multi-agent architecture in UVM?

ADVANCED

Short Answer: A multi-agent environment instantiates multiple agents (active or passive) within a single environment, each responsible for one interface. A virtual sequencer coordinates sequences across agents.

Key Steps:

1. Create separate agent classes for each interface (AXI, APB, AHB)
2. Instantiate all agents inside the environment
3. Create a virtual sequencer holding handles to all agent sequencers
4. Connect in connect_phase
5. Use virtual sequences to orchestrate multi-agent stimulus

Multi-Agent Environment

```

1 class soc_env extends uvm_env;
2   'uvm_component_utils(soc_env)
3
4   axi_agent      axi_agt;    // AXI master agent
5   apb_agent      apb_agt;    // APB config agent
6   ahb_agent      ahb_agt;    // AHB memory agent
7   soc_scoreboard sb;
8   virtual_seqr   v_seqr;     // Virtual sequencer
9

```

```

10 function void build_phase(uvm_phase phase);
11     super.build_phase(phase);
12     axi_agt = axi_agent::type_id::create("axi_agt", this);
13     apb_agt = apb_agent::type_id::create("apb_agt", this);
14     ahb_agt = ahb_agent::type_id::create("ahb_agt", this);
15     sb      = soc_scoreboard::type_id::create("sb", this);
16     v_seqr  = virtual_seqr::type_id::create("v_seqr", this);
17 endfunction
18
19 function void connect_phase(uvm_phase phase);
20     // Connect monitor analysis ports to scoreboard
21     axi_agt.monitor.ap.connect(sb.axi_imp);
22     ahb_agt.monitor.ap.connect(sb.ahb_imp);
23     // Connect virtual sequencer to agent sequencers
24     v_seqr.axi_seqr = axi_agt.sequencer;
25     v_seqr.apb_seqr = apb_agt.sequencer;
26     v_seqr.ahb_seqr = ahb_agt.sequencer;
27 endfunction
28 endclass

```

What is Layered Sequencing in UVM?

ADVANCED

Short Answer: Layered sequences separate high-level protocol transactions from low-level bus operations. An upper-layer sequence generates protocol-level transactions, and a translation layer converts them into bus-level items for the driver.

Example: Ethernet frame (upper layer) → broken into byte-level bus transactions (lower layer).

Layered Sequence Example

```

1 // Upper layer: high-level Ethernet frame
2 class eth_frame_seq extends uvm_sequence #(eth_frame);
3     task body();
4         eth_frame frame;
5         frame = eth_frame::type_id::create("frame");
6         start_item(frame);
7         frame.randomize() with {payload.size() inside {[64:1500]}};
8         finish_item(frame);
9     endtask
10 endclass
11
12 // Translation sequencer: eth_frame -> byte transactions
13 class eth_to_byte_seqr extends uvm_sequencer #(byte_txn);
14     eth_frame_seq upper_seq;
15
16     task run_phase(uvm_phase phase);
17         eth_frame frame;
18         // Pull from upper sequence, push as bytes to lower driver
19         forever begin
20             upper_seq.get_next_item(frame);

```

```

21     foreach (frame.payload[i]) begin
22         byte_txn bt = byte_txn::type_id::create("bt");
23         bt.data = frame.payload[i];
24         //Send byte to lower-level driver
25     end
26     upper_seq.item_done();
27 end
28 endtask
29 endclass

```

Explain UVM Reporting Mechanism in detail.

ADVANCED

Short Answer: UVM's reporting mechanism provides structured message logging via macros ('uvm_info, 'uvm_warning, 'uvm_error, 'uvm_fatal) with verbosity control, severity filtering, and customisable actions.

Macro	Severity	Default Action
'uvm_info	INFO	Print (if verbosity allows)
'uvm_warning	WARNING	Print + increment warning count
'uvm_error	ERROR	Print + increment error count
'uvm_fatal	FATAL	Print + stop simulation

Custom Report Server

```

1 // Custom report server to log to file
2 class file_report_server extends uvm_report_server;
3     int log_file;
4
5     function new();
6         log_file = $fopen("sim_log.txt", "w");
7     endfunction
8
9     virtual function string compose_message(
10         uvm_severity severity, string name,
11         string id, string message, string filename, int line);
12     return $sformatf("[%0t][%s][%s] %s",
13         $time, name, id, message);
14     endfunction
15 endclass
16
17 // Install custom server
18 function void build_phase(uvm_phase phase);
19     file_report_server srv = new();
20     uvm_report_server::set_server(srv);
21
22     // Set max errors before quit
23     srv.set_max_quit_count(10);
24
25     // Override error action for specific component

```

```

26     set_report_severity_action (
27         UVM_ERROR, UVM_DISPLAY | UVM_COUNT);
28 endfunction

```

How is AXI4 Protocol verified using UVM?

ADVANCED

Short Answer: AXI4 verification uses a UVM agent per AXI channel (or a combined agent with 5 channels: AW, W, B, AR, R). The driver drives master signals, monitor observes all channels, and the scoreboard verifies ordering and data integrity.

AXI4 UVM Agent Structure:

Transaction class: Contains address, data, burst type, length, size, ID

Sequences: axi4_read_seq, axi4_write_seq, axi4_burst_seq

Driver: Handles handshake (VALID/READY) on each channel

Monitor: Reconstructs full transactions from channel-level signals

Scoreboard: Verifies read data matches previously written data

AXI4 Write Transaction Sequence

```

1 class axi4_write_seq extends uvm_sequence #(axi4_txn);
2     'uvm_object_utils(axi4_write_seq)
3
4     rand bit [31:0] start_addr;
5     rand bit [7:0]  burst_len;    // 0 = 1 beat, 255 = 256 beats
6     rand bit [2:0]  burst_size;   // 0=1 B, 1=2 B, 2=4 B ...
7     rand bit [1:0]  burst_type;   // 0=FIXED, 1=INCR, 2=WRAP
8
9     constraint valid_burst {
10         burst_len inside {[0:15]};    // Max 16 beats
11         burst_size inside {2, 3};      // 4 or 8 bytes per beat
12         burst_type == 2'b01;           // INCR burst
13         start_addr[1:0] == 2'b00;      // Word-aligned
14     }
15
16     task body();
17         axi4_txn txn;
18         txn = axi4_txn::type_id::create("txn");
19         start_item(txn);
20         if(!txn.randomize() with {
21             addr      == start_addr;
22             len        == burst_len;
23             size       == burst_size;
24             burst      == burst_type;
25             txn_type   == AXI_WRITE;
26         }) 'uvm_fatal("RAND","AXI write randomization failed")
27         finish_item(txn);
28     endtask
29 endclass

```

What is UVM Testbench Reusability and how is it achieved?

ADVANCED

Short Answer: UVM testbench reusability means designing testbenches that can be reused across different projects, block levels, and chip levels with minimal changes. It is achieved through parameterisation, factory overrides, configuration objects, and modular agent design.

Reusability Strategies:

1. **Parameterised transactions and drivers** — use parameter for data width
2. **Configuration objects** — pass all settings via config objects, not hardcoded
3. **Active/Passive agent mode** — passive agents reused at chip level without driving
4. **Factory-based design** — all components replaceable via factory
5. **Package-based organisation** — group reusable components into SV packages
6. **Layered architecture** — separate protocol layer from verification layer

Reusable Agent Configuration

```

1 // Configuration object - holds all agent settings
2 class my_agent_config extends uvm_object;
3     'uvm_object_utils ( my_agent_config )
4
5     uvm_active_passive_enum  is_active = UVM_ACTIVE;
6     int unsigned             num_txns = 100;
7     bit                      checks_enable = 1;
8     bit                      coverage_enable = 1;
9     virtual my_if            vif;
10 endclass
11
12 // Agent uses config to decide behaviour
13 class my_agent extends uvm_agent;
14     my_agent_config cfg;
15
16     function void build_phase(uvm_phase phase);
17         if(! uvm_config_db #( my_agent_config ):: get(
18             this,"","cfg",cfg))
19             cfg = my_agent_config ::type_id ::create ("cfg");
20
21         // Only create driver+sequencer if active
22         if(cfg.is_active == UVM_ACTIVE) begin
23             driver      = my_driver ::type_id ::create ("driver",this);
24             sequencer    = my_sequencer ::type_id ::create ("seqr",this);
25         end
26         // Monitor always created (active or passive)
27         monitor = my_monitor ::type_id ::create ("monitor", this);
28     endfunction
29 endclass

```

How do you handle coverage-driven verification closure?

ADVANCED

Short Answer: Coverage-driven verification (CDV) uses functional coverage metrics to guide test generation, ensuring all specified scenarios are exercised. The process is: define coverage goals → run tests → measure coverage → write targeted tests for gaps → repeat.

Coverage Closure with Targeted Constraints

```

1 // Generic sequence with open constraints
2 class generic_seq extends uvm_sequence #(my_txn);
3   task body();
4     my_txn txn;
5     repeat(1000) begin
6       txn = my_txn::type_id::create("txn");
7       start_item(txn);
8       txn.randomize(); // Unconstrained - hits common cases
9       finish_item(txn);
10    end
11  endtask
12 endclass
13
14 // Targeted sequence for coverage hole: wrap burst
15 class wrap_burst_directed_seq extends uvm_sequence #(my_txn);
16   task body();
17     my_txn txn;
18     repeat(50) begin
19       txn = my_txn::type_id::create("txn");
20       start_item(txn);
21       txn.randomize() with {
22         burst_type == WRAP; // Target the gap
23         burst_len inside {1,3,7,15}; // Valid WRAP lengths
24         addr[5:0] == 6'h00; // Aligned address
25       };
26       finish_item(txn);
27     end
28   endtask
29 endclass

```

What is the difference between uvm_do and start_item/finish_item?

ADVANCED

Short Answer: 'uvm do is a convenience macro that combines create + randomize + start item + finish item in one line. start item/finish item give fine-grained control over when and how randomization occurs.

Feature	'uvm_do	start/finish_item
Control	Low (automatic)	High (manual)
Randomization	Auto	Manual, with constraints
Pre-randomize	Not possible	Possible between calls
Reuse object	Creates new	Can reuse existing

uvm_do vs start_item/finish_item

```

1 // Using 'uvm_do (simple, less control)
2 task body_simple ();
3     my_txn txn;
4     'uvm_do (txn) // Create+rand+send in 1 line
5     'uvm_do_with (txn, {txn.data == 8'hFF;}) // With constraints
6 endtask
7
8 // Using start_item/finish_item (full control)
9 task body_controlled ();
10     my_txn txn;
11     txn = my_txn::type_id::create ("txn");
12
13     start_item (txn);
14     // Can modify txn here BEFORE randomization
15     txn.addr = calculate_next_addr ();
16
17     // Randomize with inline constraint
18     if (!txn.randomize () with {
19         data inside {[8'hA0:8'hFF]};
20         valid == 1;
21     }) 'uvm_fatal ("RAND", "Failed ")
22
23     // Can inspect txn here BEFORE sending
24     'uvm_info ("SEQ", $sformatf ("Sending: %s", txn.sprint ()), UVM_HIGH )
25
26     finish_item (txn);
27     // Execution resumes here after driver calls item_done ()
28 endtask

```

How do you verify APB protocol using UVM?

ADVANCED

Short Answer: APB (Advanced Peripheral Bus) verification in UVM involves a dedicated APB agent with a driver handling the 3-phase APB protocol (IDLE → SETUP → ENABLE), monitor capturing transactions, and scoreboard checking read-after-write consistency.

APB Driver — 3-Phase Protocol

```

1 class apb_driver extends uvm_driver #(apb_txn);
2   'uvm_component_utils ( apb_driver )
3   virtual apb_if vif;
4
5   task run_phase (uvm_phase phase);
6     apb_txn txn;
7     // Initialize APB to IDLE state
8     vif.psel    <= 0;
9     vif.penable <= 0;
10    forever begin
11      seq_item_port.get_next_item ( txn );
12      drive_apb ( txn );
13      seq_item_port.item_done ();
14    end
15  endtask
16
17  task drive_apb ( apb_txn txn );
18    // SETUP phase: assert PSEL, deassert PENABLE
19    @(posedge vif.pclk);
20    vif.psel    <= 1;
21    vif.paddr    <= txn.paddr;
22    vif.pwrite    <= txn.pwrite;
23    vif.pwdata    <= txn.pwrite ? txn.pwdata : '0;
24    vif.penable <= 0;
25
26    // ENABLE phase: assert PENABLE
27    @(posedge vif.pclk);
28    vif.penable <= 1;
29
30    // Wait for PREADY (slave acknowledges)
31    do @(posedge vif.pclk); while (!vif.pready);
32
33    // Capture read data
34    if (!txn.pwrite)
35      txn.prdata = vif.prdata;
36
37    // Return to IDLE
38    vif.psel    <= 0;
39    vif.penable <= 0;
40  endtask
41 endclass

```

What is constraint solving and how do you debug it?**ADVANCED**

Short Answer: SystemVerilog's constraint solver automatically generates values satisfying all active constraints. Debugging constraint failures involves isolating constraints, using `constraint mode()`, and enabling solver debug options.

Constraint Debugging Techniques

```
1 class complex_txn extends uvm_sequence_item ;
2   rand bit [7:0]   addr;
3   rand bit [31:0]  data;
4   rand bit [2:0]   burst_len;
5   rand bit         valid;
6
7   // Constraint blocks
8   constraint c_addr { addr inside {[8'h10:8'h7F]}; }
9   constraint c_data { data != 0; }
10  constraint c_burst { burst_len inside {1,3,7}; }
11  constraint c_valid { valid == 1; }
12
13  // Debug technique 1: Disable constraints one by one
14  task debug_randomize();
15    // Turn off specific constraints
16    c_burst.constraint_mode(0); // Disable burst constraint
17    if(!this.randomize())
18      'uvm_error("RAND","Still failing without burst constraint!")
19    c_burst.constraint_mode(1); // Re-enable
20
21    // Turn off ALL constraints
22    this.constraint_mode(0);
23    if(!this.randomize())
24      'uvm_fatal("RAND","Fails even with no constraints!")
25    this.constraint_mode(1);
26  endtask
27 endclass
28
29 // Debug technique 2: Use solve...before for ordering
30 class ordered_txn extends uvm_sequence_item;
31   rand bit [3:0] len;
32   rand bit [3:0] data[];
33
34   constraint c_order {
35     solve len before data; // Solve len first, then data
36     data.size() == len;
37   }
38 endclass
39
40 // Debug technique 3: Inline soft constraints
41 class soft_txn extends uvm_sequence_item;
42   rand bit [7:0] priority_level;
43   constraint c_soft {
44     soft priority_level == 8'hFF; // Preferred but overridable
45   }
46 endclass
```

How do you optimise UVM simulation performance?**ADVANCED**

Short Answer: UVM simulation performance can be significantly improved through verbosity control, disabling unused features, optimising coverage sampling, using the factory judiciously, and leveraging tool-specific acceleration.

UVM Performance Optimisation

```

1 // 1. Set verbosity to LOW for regression runs
2 // Command line: +UVM_VERBOSITY=UVM_LOW
3
4 // 2. Disable coverage in directed tests
5 function void build_phase(uvm_phase phase);
6     uvm_config_db #(bit)::set(this, "*", "coverage_enable", 0);
7 endfunction
8
9 // 3. Avoid string operations in hot paths
10 // BAD - string formatting in tight loop:
11 task run_phase(uvm_phase phase);
12     forever begin
13         seq_item_port.get_next_item(txn);
14         // This creates strings every transaction - slow!
15         'uvm_info("DRV", $sformatf("Driving %s", txn.sprint()),
16                 UVM_HIGH)
17         drive_txn(txn);
18         seq_item_port.item_done();
19     end
20 endtask
21
22 // GOOD - check verbosity before formatting:
23 task run_phase_optimised(uvm_phase phase);
24     forever begin
25         seq_item_port.get_next_item(txn);
26         if(get_report_verbosity_level() >= UVM_HIGH)
27             'uvm_info("DRV", $sformatf("Driving %s", txn.sprint()),
28                     UVM_HIGH)
29         drive_txn(txn);
30         seq_item_port.item_done();
31     end
32 endtask
33
34 // 4. Sample coverage conditionally
35 function void write(my_txn txn);
36     if(coverage_enable)
37         cg.sample(); // Don't sample in every run
38 endfunction
39
40 // 5. Use uvm_pool for object reuse
41 // Instead of creating new objects every transaction,
42 // reuse from a pool to reduce GC pressure
43 class txn_pool;
44     static my_txn pool[$];
45     static function my_txn get();
46         if(pool.size() > 0) return pool.pop_back();
47         return my_txn::type_id::create("txn");
48     endfunction
49     static function void put(my_txn txn);
50         pool.push_back(txn);
51     endfunction
52 endclass

```

```
50 endclass
```

Key Takeaways

Set verbosity to UVM LOW in regression; only enable HIGH for debug runs
Disable coverage and checks for performance runs, enable for closure runs
Avoid \$sprintf in tight simulation loops — expensive string operations
Use tool-level optimisations: VCS +vcs+flush+all, Questa -O5
Profile simulation with tool's built-in profiler before optimising

6 Quick Reference: UVM Cheat Sheet

6.1 UVM Macro Reference

Macro	Purpose
<code>`uvm_component_utils(T)</code>	Register component T with factory
<code>`uvm_object_utils(T)</code>	Register object T with factory
<code>`uvm_info(ID,MSG,VERB)</code>	Print info message
<code>`uvm_error(ID,MSG)</code>	Print error message
<code>`uvm_fatal(ID,MSG)</code>	Print fatal + stop simulation
<code>`uvm_do(item)</code>	Create + randomize + send item
<code>`uvm_do_with(item,{c/})</code>	uvm_do with inline constraint
<code>`uvm_field_int(F,FLAG)</code>	Field automation (print/copy/compare)
<code>`uvm_declare_p_sequencer</code>	Declare typed p sequencer handle
<code>`uvm_register_cb(C,CB)</code>	Register callback with component
<code>`uvm_do_callbacks(C,CB,F)</code>	Execute registered callbacks

Table 1. Essential UVM Macros Quick Reference

6.2 Interview Preparation Checklist

Interview Readiness Checklist

- ☐ Can explain UVM architecture without looking at notes
- ☐ Can write a complete driver from scratch
- ☐ Understand factory override (type + instance)
- ☐ Know objection mechanism inside-out
- ☐ Can explain TLM ports and analysis ports
- ☐ Understand virtual interface passing via config-db
- ☐ Know difference between uvm-object and uvm-component
- ☐ Can implement a basic scoreboard
- ☐ Understand UVM RAL and adapter concept
- ☐ Can write a virtual sequence for multi-agent env
- ☐ Know at least one protocol (AXI/APB/AHB) in UVM context
- ☐ Can debug a deadlock in sequence-driver handshake
- ☐ Understand phase execution order (top-down vs bottom-up)
- ☐ Know how to use +UVM-VERBOSITY and print-topology

Closing Note from VeriCore

Thank You for Choosing VeriCore!

This document has been carefully prepared to help VLSI verification engineers at all levels — freshers, 1–3 year experienced engineers, and those looking to transition into ASIC verification roles.

Key Reminders:

- Practice writing UVM code daily — reading is not enough
- Set up a free Cadence Xcelium / Synopsys VCS / Mentor Questa environment
- Build a small UVM testbench project for your GitHub portfolio
- Follow VLSI communities on LinkedIn for the latest industry trends
- Revise all 30 questions at least 3 times before your interview

For doubts, questions, or feedback:

DM **"PDF"** on our channel and we will answer all your questions personally.

Wishing you great success in your VLSI verification career!

VeriCore

ASIC Verification Series | SystemVerilog | UVM | DFT

Qc 2025 VeriCore. For Educational Purposes Only.